Absolutely. The easiest way to understand **Parallel GC** is:

> **Parallel GC uses essentially the same generational strategy as Serial GC, but uses multiple GC worker threads to perform collection in parallel.**

Its primary goal is **high application throughput**, rather than extremely low GC pause times.

---

1. What is Parallel GC?

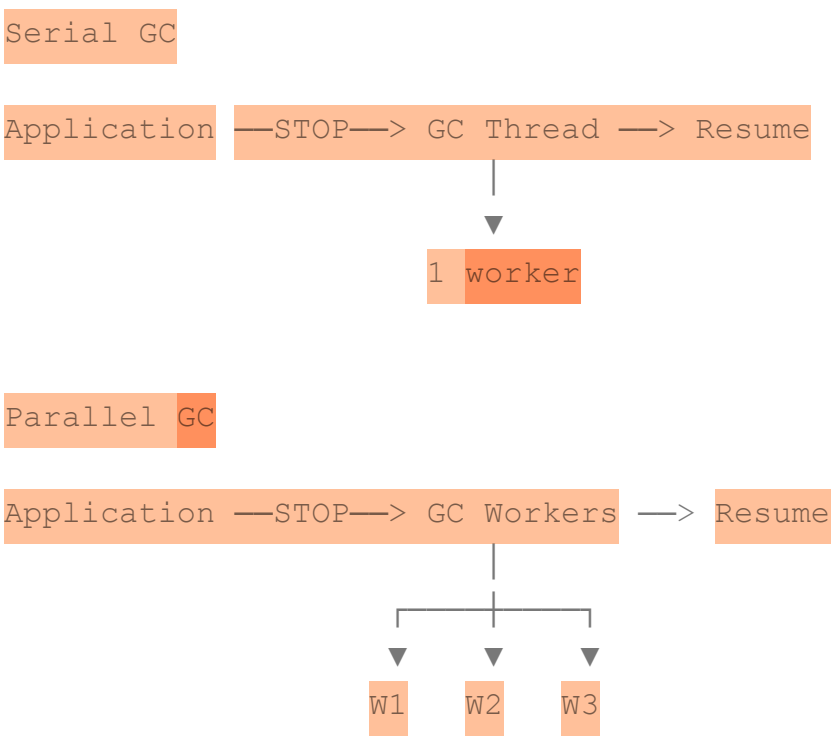
In HotSpot, Parallel GC is enabled with:

```
java -XX:+UseParallelGC MyApplication
```

It is:

- **Generational**
- **Stop-the-world**
- **Multi-threaded during GC**
- Young generation: **parallel copying**
- Old generation: **parallel mark + summary/compaction**
- Designed primarily for **throughput**

The most important distinction:



The application is still stopped.

"Parallel" refers to the GC work, not concurrent execution with the application.

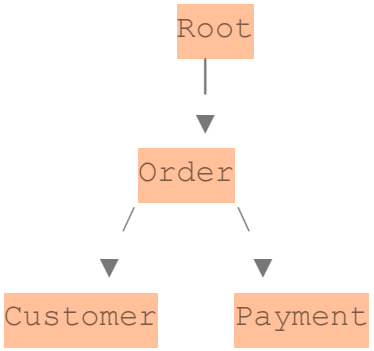
---

2. Example Application

Suppose our application creates many objects:

```
Order order = new Order();
Customer customer = new Customer();
Payment payment = new Payment();
```

Conceptually:



Meanwhile, the application creates thousands of temporary objects.

Eventually Eden fills:

```
Young Generation

Eden
┌─────────────────────────────────────────┐
│ A B C D E F G H I J K L M N O ...       │
└─────────────────────────────────────────┘

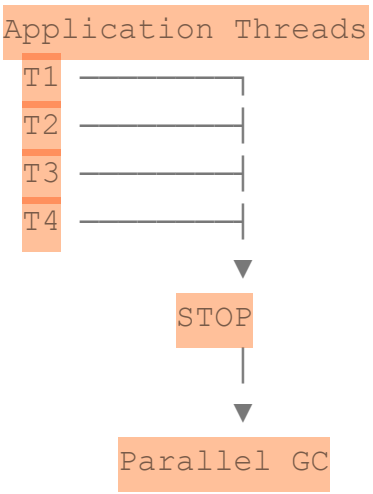
                ↓
              FULL
```

Parallel GC starts a Young GC.

---

3. Stop the Application

First:



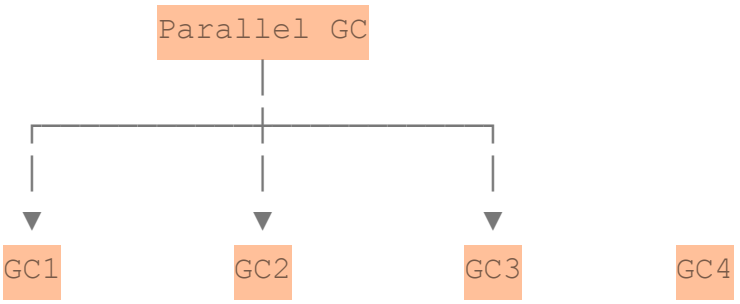
No Java application thread modifies the heap while this collection is happening.

This is fundamentally different from G1/Shenandoah/ZGC, which perform substantial GC work concurrently with application threads.

---

4. Multiple GC Workers Start

Suppose the JVM decides to use four GC workers:



They divide the collection work.

Imagine Eden contains:



The workers can process different portions/work units:

```
GC1 → A B C D E
GC2 → F G H I J
GC3 → K L M N O
GC4 → P Q R S T
```

The actual implementation uses more sophisticated work partitioning and work stealing; it isn't simply a fixed contiguous split.

---

5. Find Live Objects

Suppose the root graph looks like:

```
Root
 ├── A
 ├── F
 └── K
```

```
A → C
F → H
K → M
```

Therefore:

```
Live:
A C F H K M
```

```
Dead:
B D E G I J L N O P Q R S T
```

The GC workers cooperate to identify and process the live objects.

---

6. Copy Live Objects

Just like Serial GC, Parallel GC uses a copying strategy for the young generation.

Suppose:

```
Eden:
A B C D E F G H I J K L ...
```

After collection:

```
                Survivor Space
┌──────────────────────────────────────┐
│ A C F H K M                          │
└──────────────────────────────────────┘
```

Only the live objects need to be copied.

The GC workers can perform those operations concurrently with each other:

```
GC1 ── copy A, C ──────┐
                       │
GC2 ── copy F, H ──────┤
                       ├──> Survivor
GC3 ── copy K, M ──────┤
                       │
GC4 ── other work ─────┘
```

---

7. What About Object References?

Suppose:

```
A → C
```

and A is copied to another memory location:

```
Before:

A @ 0x1000
 │
 └──→ C @ 0x2000
```

After copying:

```
A' @ 0x5000
  |
  └──→ C' @ 0x6000
```

The GC must update references to the new locations.

Because the application is stopped, the collector can perform this relocation and reference adjustment without racing with mutator threads.

This is one of the advantages of STW compaction.

---

8. Survivor Spaces

Just like Serial GC, Parallel GC uses two Survivor spaces.

For example:

```
Before GC:

Eden ————————> From Survivor ————————> To Survivor
[A B C D E]       [F G H]                [empty]
```

After GC:

```
Eden = empty

From Survivor = empty

To Survivor:
[A C F H]
```

Then their roles swap:

```
To → From
From → To
```

The next Young GC will copy surviving objects into the other Survivor space.

---

9. Object Promotion

Suppose an object survives several collections.

Conceptually:

```
GC #1 → age 1
GC #2 → age 2
GC #3 → age 3
...
```

Eventually it may be promoted:

```
Survivor
   |
   | promotion
   ▼
Old Generation
```

For example:

```
Young:

S0
┌──────────────┐
| A(age=10)    |
| C(age=10)    |
└──────────────┘
       |
       ▼

Old Generation

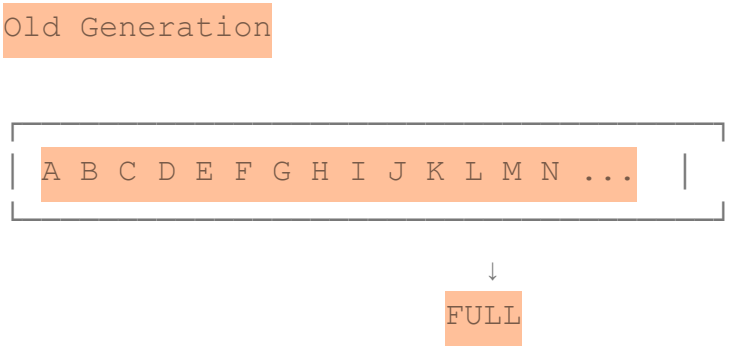
┌──────────────────────────┐
| X Y Z A C                |
└──────────────────────────┘
```

Again, actual Parallel GC promotion decisions are more sophisticated than simply "age reaches N."

---

10. What Happens When Old Gen Gets Full?

Eventually:



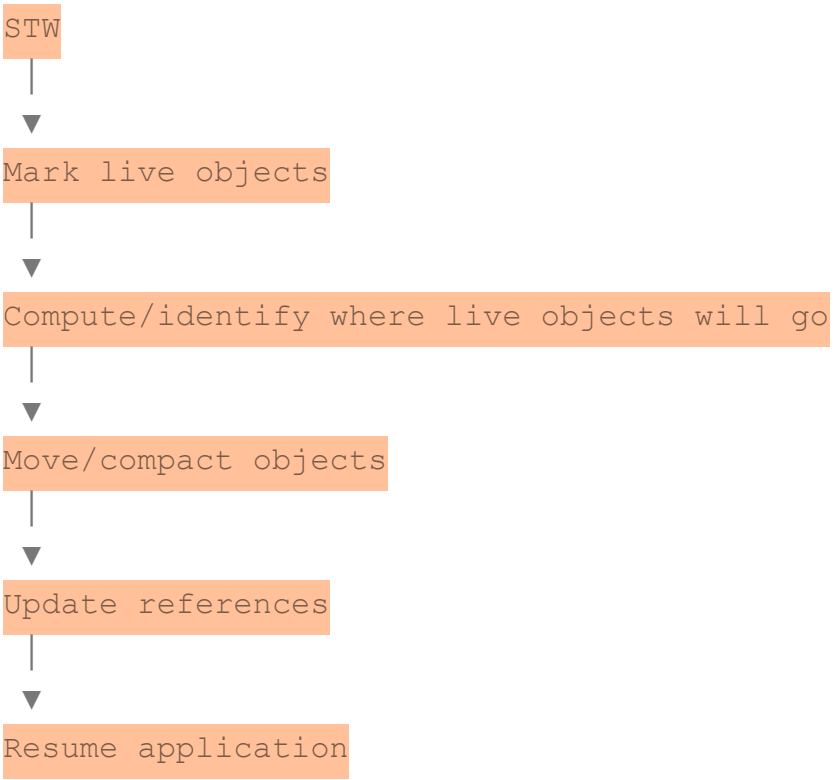
Parallel GC needs an old-generation collection.

This is also **STW**.

---

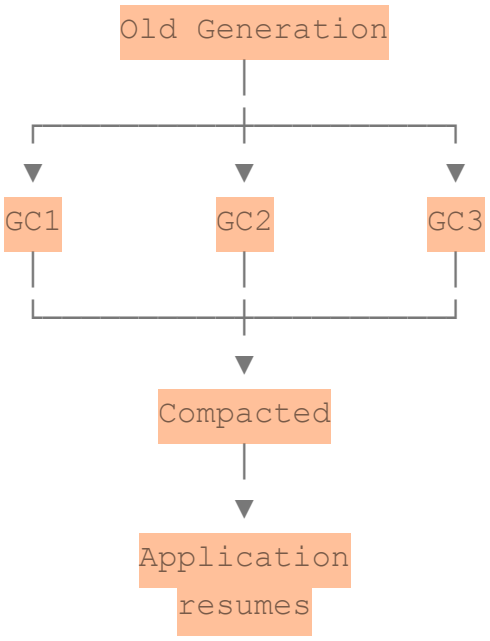
11. Parallel Old GC

Conceptually, the process is:



The work is performed by multiple GC threads.

For example:



---

12. Why Compaction?

Suppose before collection:

```
Old Gen:

┌──────────────────────────────────────────┐
| A |   | B |   |   | C |   | D |        |
└──────────────────────────────────────────┘

   ↑       ↑       ↑       ↑
 dead    dead    dead    dead
```

There is free space, but it is fragmented.

After compaction:

```
┌──────────────────────────────────────────┐
| A | B | C | D | FREE FREE FREE FREE |
└──────────────────────────────────────────┘
```

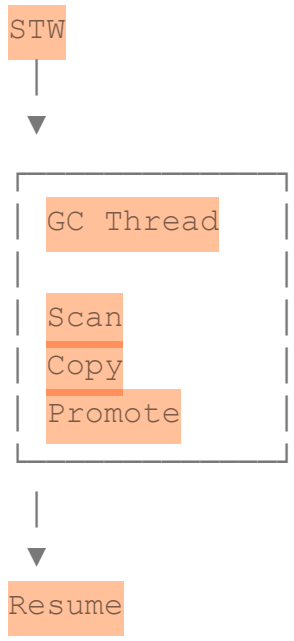
This provides a large contiguous allocation area.

---

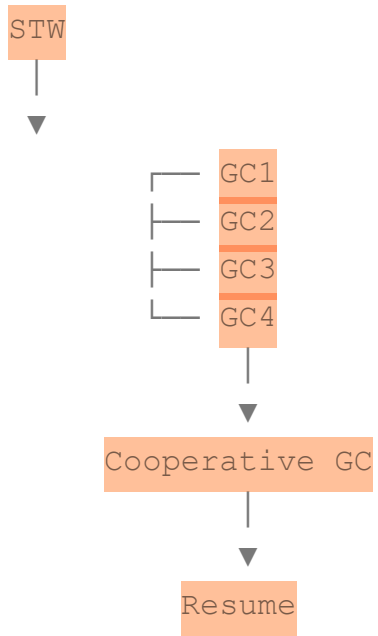
13. The Key Difference from Serial GC

Let's compare a Young GC.

Serial GC



Parallel GC



So if a collection requires:

```
Single-threaded:
        400 ms

4 GC workers:
        ~100–150 ms
```

the application can potentially resume much sooner.

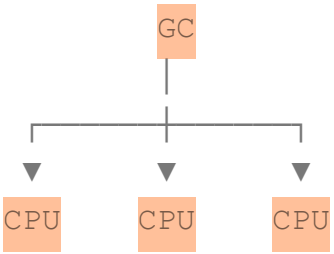
The exact improvement depends heavily on heap size, live-set size, CPU, object graph topology, GC configuration, and memory bandwidth.

---

14. But There Is a Trade-off

More GC threads don't mean unlimited scalability.

Imagine:



GC competes for:

- CPU
- memory bandwidth
- cache
- memory subsystem resources

Eventually:

```
More GC threads
        ▼
Less useful parallelism
        ▼
CPU/memory contention
```

So the JVM dynamically chooses an appropriate number of GC workers.

You can influence it with options such as:

```
-XX:ParallelGCThreads=N
```

---

15. Why Parallel GC Is Called a Throughput Collector

Suppose an application runs for 100 seconds.

Serial GC

```
Application execution: 90 sec
GC:                    10 sec

Throughput ≈ 90%
```

Parallel GC might reduce GC time:

```
Application execution: 97 sec
GC:                    3 sec

Throughput ≈ 97%
```

The exact numbers are illustrative.

The key objective is:

> **Spend as little total CPU/time as possible doing GC so that the application gets maximum execution time.**

This is different from a low-latency collector.

---

16. Parallel GC vs G1

This distinction is extremely important for your MAANG-level JVM interviews.

	Parallel GC	G1
STW	Yes	Some phases
GC workers	Multiple	Multiple
Main goal	**Throughput**	**Latency + throughput balance**
Heap organization	Generational	Region-based generational
Young GC	STW	STW
Concurrent marking	No	Yes
Concurrent application/GC work	Limited/no	Yes, for marking phases
Compaction	STW	Evacuation
Large heap suitability	Good	Better
Pause-time targeting	Not primary goal	Yes

The crucial conceptual difference is:

```
Parallel GC:

"Stop the application and finish GC as quickly
 as possible using many CPUs."


G1:

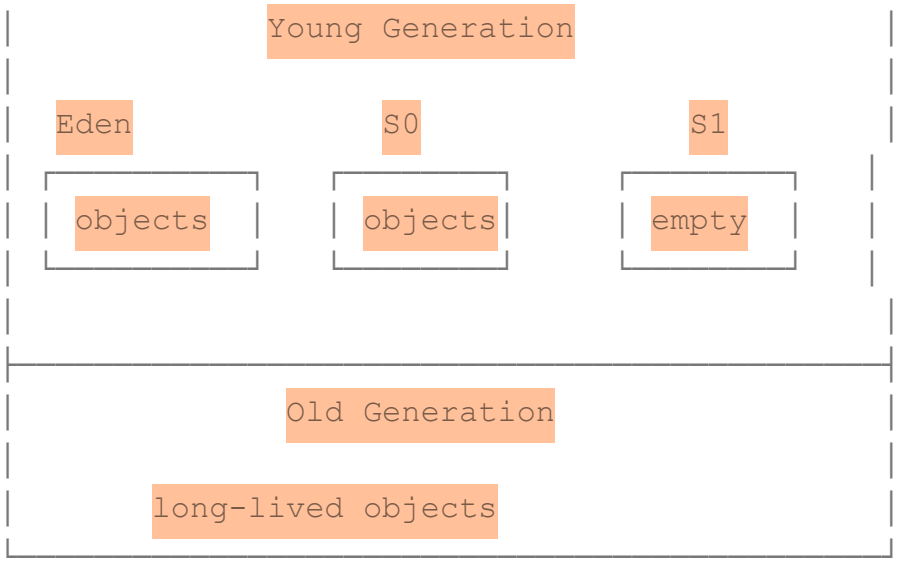
"Do enough GC concurrently and select regions
 intelligently so that pauses stay manageable."
```

---

17. A Complete Example

Imagine a 4 GB heap:

```
                    4 GB Heap
┌──────────────────────────────────────────────────┐
```



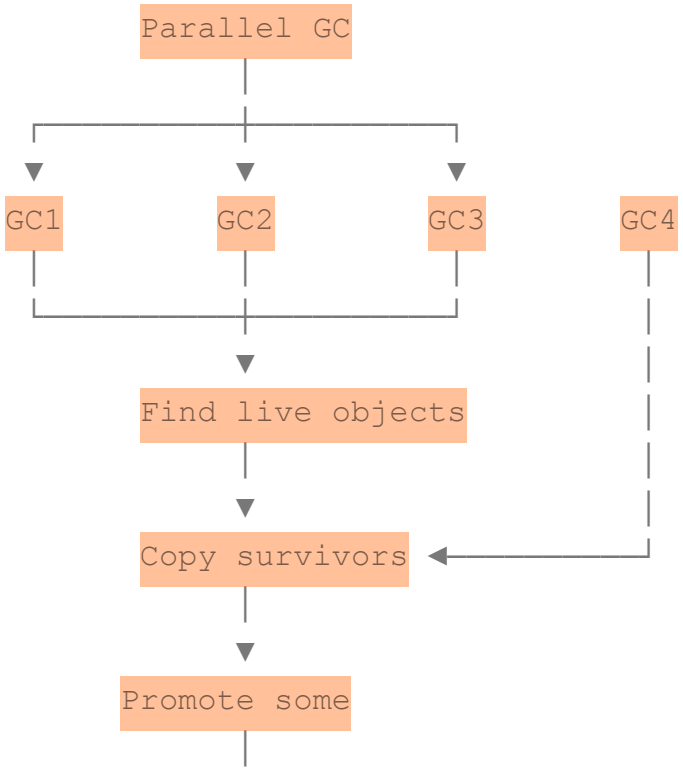
Eden fills:

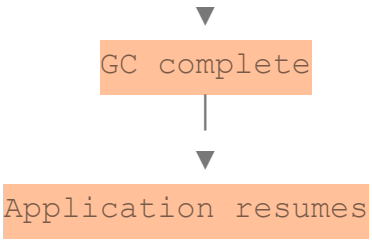
```
Eden → FULL
```

Application threads stop:

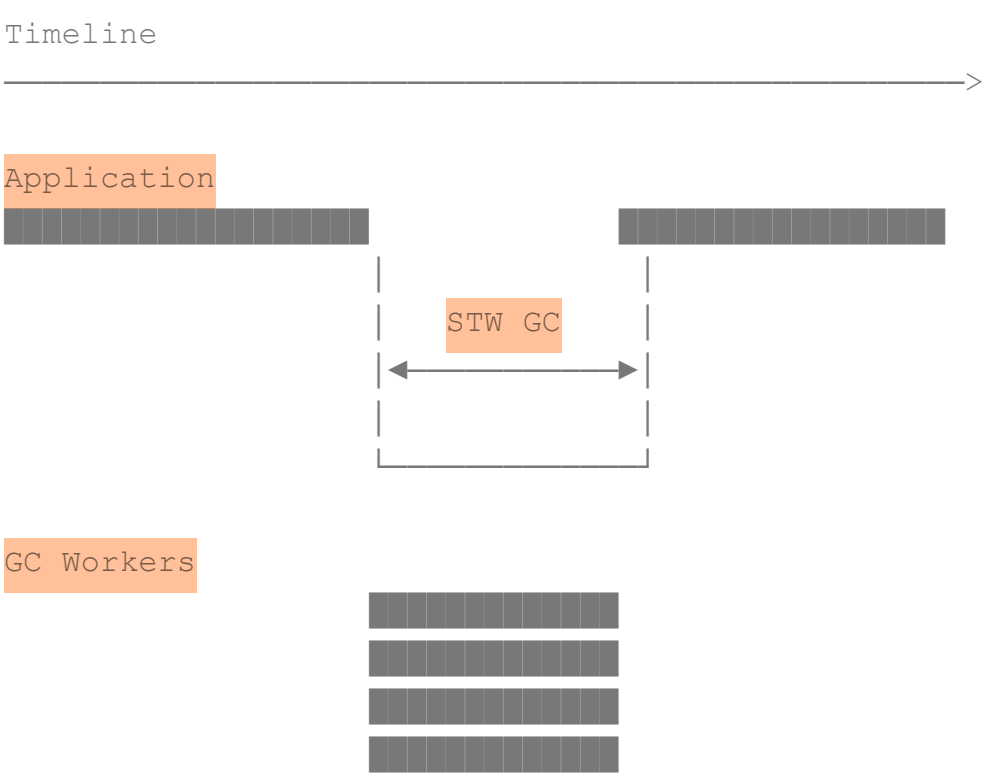
```
T1 ── X
T2 ── X
T3 ── X
T4 ── X
```

GC workers start:





The entire pause might conceptually look like:

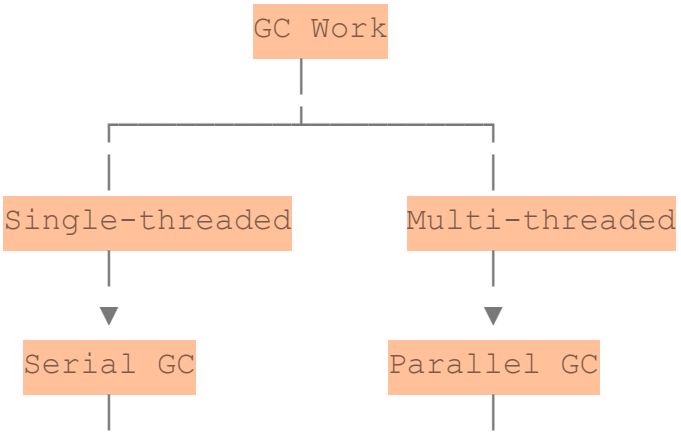


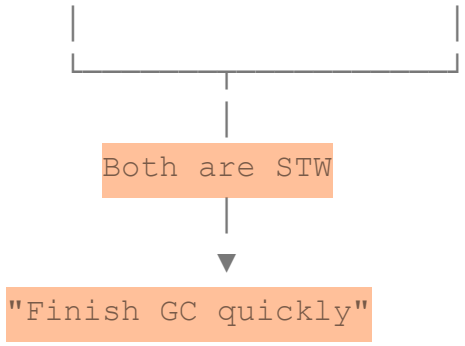
The key is that **the GC workers run in parallel, but the application does not run during that interval.**

---

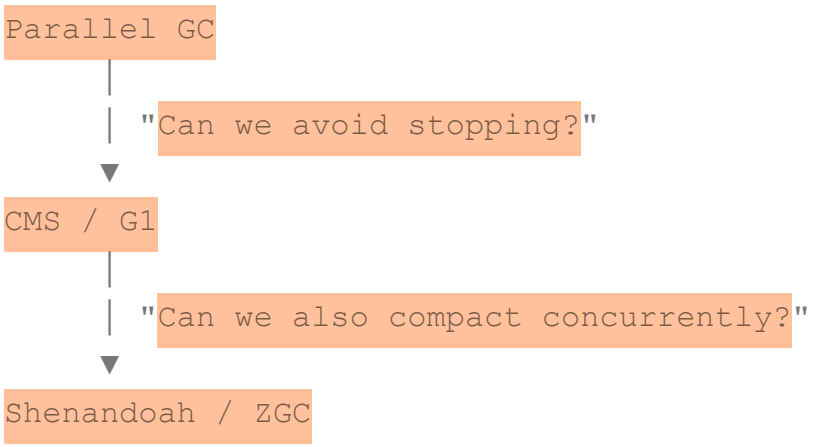
18. Interview Mental Model

Think of the collectors this way:





Then:



That gives you a useful evolution of JVM GC design:

> **Serial → parallelize GC work → perform marking concurrently → perform relocation/compaction concurrently.**

The interview answer to memorize

> **"Parallel GC is a generational, stop-the-world collector optimized for throughput. During a GC pause, multiple GC worker threads cooperate to scan roots, identify live objects, copy surviving young objects, promote objects, and compact the old generation. Compared with Serial GC, it reduces GC pause duration by exploiting multiple CPU cores, but the application remains paused during the collection."**

The next logical step is **CMS**, because it introduces the major conceptual change: **the application is allowed to run while GC is marking and sweeping**.